Napredni Kotlin

Nasljeđivanje, Sučelja, Funkcije višeg reda, Generics, Kolekcije, Delegati

Luka Kordić
Android @Spyrosoft
GDG Osijek

Nasljeđivanje

Nasljeđivanje

- Roditeljska klasa dijeli svojstva i metode s izvedenom klasom.
- Izvedena klasa može naslijediti samo jednu roditeljsku.
- Kotlin klase su po zadanim postavkama final (ne mogu se nasljeđivati).

```kotlin
open class Person(
    val name: String,
    val lastName: String
) {
```

```kotlin
class Teacher(
    name: String,
    lastName: String,
    salary: Double
) : Worker(name, lastName,
Job("Teacher", salary))
```

```kotlin
open class Worker(
    name: String,
    lastName: String,
    protected val job: Job
) : Person(name, lastName)
```

```
open class Person(
    val name: String,
    val lastName: String
) {
```

```
class Teacher(
    name: String,
    lastName: String,
    salary: Double
) : Worker(name, lastName,
Job("Teacher", salary))
```

```
open class Worker(
    name: String,
    lastName: String,
    protected val job: Job
) : Person(name, lastName)
```

```
open class Person(
    val name: String,
    val lastName: String
) {
```

```
class Teacher(
    name: String,
    lastName: String,
    salary: Double
) : Worker(name, lastName,
Job("Teacher", salary))
```

```
open class Worker(
    name: String,
    lastName: String,
    protected val job: Job
) : Person(name, lastName)
```

Nasljeđivanje

- Open ključna riječ - klase su zatvorene po defaultu.
- Nasljeđivanjem stvaramo hijerarhiju klasa.
- Nasljeđuju se samo metode i atributi koji nisu privatni.
- protected - ograničava vidljivost samo na izvedene klase.

Sučelja

Sučelja

- Slična klasama, ali uvijek apstraktna.
- “Ugovor” koji se mora poštivati.
- Apstraktne metode se moraju implementirati.
- Klase mogu implementirati bilo koji broj sučelja.

Sučelja

```
interface OnClickListener {
    fun onClick()
}

interface OnLongClickListener {
    fun onLongClick()
}
```

Sučelja

```
interface CombinedClickListener : OnClickListener, OnLongClickListener
```

Sučelja

```
interface CombinedClickListener : OnClickListener, OnLongClickListener

class CustomListener : CombinedClickListener {
    override fun onClick() {
        // Implement
    }
    override fun onLongClick() {
        // Implement
    }
}
```

Apstrakcija i polimorfizam

Apstrakcija i polimorfizam

- Sposobnost tipa da primi više oblika.
- Možemo koristiti zajednički (apstraktniji) tip za neko ponašanje bez da znamo konkretnu implementaciju.

```kotlin
interface Attacker {
    fun attack()
}

class Swordsman: Attacker {
    override fun attack() {
        println("Swinging a sword...")
    }
}

class Archer: Attacker {
    override fun attack() {
        println("Drawing a bow...")
    }
}

class Dragon: Attacker {
    override fun attack() {
        println("Breathing fire...")
    }
}
```

```
interface Attacker {
    fun attack()
}

class Swordsman: Attacker {
    override fun attack() {
        println("Swinging a sword...")
    }
}

class Archer: Attacker {
    override fun attack() {
        println("Drawing a bow...")
    }
}

class Dragon: Attacker {
    override fun attack() {
        println("Breathing fire...")
    }
}
```

```kotlin
class Boss(private val character: Attacker) {
    fun attackPlayer() {
        character.attack()
    }
}

fun main() {
    val boss = Boss(Archer())
    println(boss.attackPlayer())
}
```

Output?

```
Drawing a bow...
```

Apstrakcija i polimorfizam

- Mnogi design patterni koriste polimorfizam
- Smanjuje kompleksnost koda
- Jednostavnije sintaksa i čitkiji kod

Override & overload

Override

- Mijenjamo implementaciju superklase
- override fun functionName()

```kotlin
open class Worker(
    name: String,
    lastName: String,
    protected val job: Job
) : Person(name, lastName) {

    override fun toString(): String {
        return "My name is $name $lastName. I work as a ${job.name}"
    }
}
```

```kotlin
open class Worker(
    name: String,
    lastName: String,
    protected val job: Job
) : Person(name, lastName) {

    override fun toString(): String {
        return "My name is $name $lastName. I work as a ${job.name}"
    }
}
```

Overload

- Isto ime
- Drugačiji potpis funkcije

```
fun doWork() {
    // Do something
}

fun doWork(assistant: Worker) {
    // Do something with the assistant’s help
}
```

Različiti parametri

```
fun sum(float1: Float, float2: Float): Float {
    return float1 + float2
}

fun sum(int1: Int, int2: Int): Int {
    return int1 + int2
}
```

Različiti tipovi

1. Zadatak

Modelirajte nekoliko pametnih uređaja, npr. pametna žarulja i pametni hladnjak. Oba uređaja trebaju imati ime i trenutno stanje kao svojstva te sposobnost uključivanja i isključivanja. Pametna žarulja treba imati sposobnost prigušivanja svjetla.

Napišite kontroler koji može upravljati tim uređajima, uključujući funkcionalnost za uključivanje, isključivanje
i prigušivanje svjetla (ako je primjenjivo).

Koristite apstraktnu klasu, sučelje i prepisivanje funkcija.

Funkcije višeg reda

Funkcije višeg reda

- Možemo ih pohraniti u varijable.
- Možemo ih slati kao argumente drugim funkcijama.
- Možemo ih vratiti kao rezultat funkcijskog poziva.

Funkcijski tipovi

- Tip bez parametara i povratnim tipom Unit

```
val noParamFunType: () -> Unit
```

Funkcijski tipovi

- Tip sa String parametrom i Unit povratnim tipom

```
val oneParamFunType: (String) -> Unit
```

Funkcijski tipovi

- Tip sa String parametrom i String povratnim tipom

```
val funWithReturnType: (String) -> String
```

Funkcijski izrazi

- Funkcije koje nisu deklarirane nego se prosljeđuju izravno kao izraz.
- Vrijednosti funkcijskih tipova
  - Lambda `{ a, b -> a + b }`
  - Anonimna funkcija `fun(a, b): Int = a - b`

Funkcijski tip

- `() -> Unit`
- `(String) -> Boolean`
- `(Int, Int) -> Int`
- `String.() -> Boolean`

Funkcijski izraz

- `{  }`
- `{ it.contains('a') }`
- `{ a, b -> a + b }`
- `{ this.contains('a') }`

Lambde nemaju return ključnu riječ. Zadnji izraz se vraća.

Funkcije višeg reda

- Callbacks
- Kolekcije
- Jetpack Compose

Inline modifikator

```
fun main() {
    runSomeCode {
        println("Hello there!")
    }
}

inline fun runSomeCode(codeToRun: () -> Unit) {
    codeToRun()
}
```

Inline modifikator

```
fun main() {
    println("Hello there!")
}
```

Extension functions

Extension functions

- Dodavanje funkcionalnosti tipovima kojima nemamo pristup.
- Mogu se koristiti na svim instancama nekog tipa.

Extension functions

```
fun Int.squared(): Int = this * this


fun main() {
    println(5.squared())
}
```

```
suspend fun <T : Mappable<R>, R : Any> Call<T>.awaitResult(): Result<R> {
   return suspendCoroutine { continuation ->

       enqueue(object : Callback<T> {
           override fun onFailure(call: Call<T>?, error: Throwable?) {
               continuation.resume(Failure(error))
           }

           override fun onResponse(call: Call<T>?, response: Response<T>?) {
               response?.body()?.run { continuation.resume((mapToData())) }
                         response?.errorBody()?.run {
     continuation.resume(Failure(HttpException(response))) }
           }
       })
   }
}
```

```
apiService.getWeather(cityName, API_KEY).awaitResult()
```

2. Zadatak

- Napišite funkciju operate koja prima dva broja i funkcijski tip koji određuje matematičku operaciju. Pozovite funkciju da odradi zbrajanje, množenje i pronađe veći od dva predana argumenta.

- Napišite ekstenziju na klasi String koja provjerava je li predani string email. String je email ako ima znak ‘@’ i ‘.’.

Generics

Generics

- Definiramo ponašanje koje se može primjeniti na različite tipove.
- Klase, sučelja i funkcije mogu biti generični.

```kotlin
val listOfNames = listOf(
    "Filip",
    "Marin",
    "Damir",
    "Luka",
    "Tomislav"
)
```

```
public fun <T> listOf(vararg elements: T): List<T> =
    if (elements.size > 0) {
        elements.asList()
    } else{
        emptyList()
    }
```

Pauza! 10 min

Kolekcije

Collections

- Kolekcija objekata istog tipa.
- List, Set, Map, Array...
- Promjenjive i nepromjenjive kolekcije.
- Mnoge ugrađene funkcije spremne za korištenje.

List

- Generična poredana kolekcija elemenata.

```
public interface List<out E> : Collection<E>
```

List

```
val teachers = listOf("Bruno", "Luka", "Filip",
"Goran")

teachers.run {
   println(count())
   println(first())
   println(find { it.startsWith('L') })
}
```

Array

- Sadrži fiksan broj vrijednosti istog tipa.
- Ako imate posebne memorijske zahtjeve u aplikaciji ili optimizirate performanse izvođenja programa.

```
val teachers = arrayOf("Bruno", "Luka", "Filip",
"Goran")
teachers.forEach { println(it) }
```

Array

```
val integers: Array<Int> = arrayOf(1, 2, 3) // Boxed
objects -> Integer

val ints: IntArray = intArrayOf(1, 2, 3) //
primitive type -> int
```

Set

- Generična neporedana kolekcija elemenata.
- Ne podržava duplikate.

```
public interface Set<out E> : Collection<E>
```

Set

```
val teachers = setOf("Bruno", "Luka", "Filip",
"Goran", "Luka", "Filip")
println(teachers)
```

```
Output: [Bruno, Luka, Filip, Goran]
```

Map

- Sadrži ključ-vrijednost parove.
- Ključevi su jedinstveni.
- Svaki ključ može imati jednu vrijednost.

```
public interface Map<K, out V>
```

Map

```
val nameToJob = mapOf(
    "Filip" to "Compose Dev",
    "Luka" to "Android Dev"
)

val nameJobPair: Pair<String, String> = "Bruno" to
"Professor"
```

(Ne)Promjenjivost

(Ne)Promjenjivost

```
val myList = listOf(1, 2, 3)
myList.add(4)
```

Ne postoji add metoda na nepromjenjivoj listi.

(Ne)Promjenjivost

```
val myList = mutableListOf(1, 2, 3)
  myList.add(4)
  myList.remove(1)
  println(myList)

 // [2, 3, 4]
```

(Ne)Promjenjivost

```
val myList = mutableSetOf(1, 2, 3)
  myList.add(4)
  myList.remove(1)
  println(myList)

 // [2, 3, 4]
```

(Ne)Promjenjivost

```
val myMap = mutableMapOf(
    1 to "Luka",
    2 to "Filip",
    3 to "Bruno"
)
myMap.put(4, "Goran")
myMap[5] = "David"
myMap.remove(1)
```

3. Zadatak

- Definirajte nepromjenjivu listu brojeva koja predstavlja dob ljudi koji žele ući u klub. Napišite program koji radi kao izbacivač i filtrira one koji su mlađi od 18 god. Također, ispišite dob svake osobe koja može ući u klub.

- Definirajte listu imena.
  - Grupirajte imena prema prvom slovu.
  - Za svako slovo ispišite koja imena su mu dodijeljena.
  - Pronađite koje slovo ima najviše imena.
  - Ispišite broj imena za svako slovo.

Data klase

Data klase

- Služe za držanje i prijenos podataka
- Kompajler generira metode za nas
  - copy, toString, equals, hashCode...
- Označavaju se data modifikatorom

Data classes

```
data class User(
    val name: String,
    val lastName: String,
    val job: String,
)
```

Data classes

```
val filip = User("Filip", "Babic",
"Compose Dev")

val filipClone = filip.copy()
```

Data classes

```
val (name, lastName, job) = filip

// Destrukturiranje
```

Data classes

```
println(filip == filipClone)
   true

println(filip)
      User(name=Filip, lastName=Babic,
   job=Compose Dev)
```

Sealed klase

Sealed klase

- Klase definirane s konačnim brojem tipova.
- Mogu biti definirane samo unutar iste datoteke.
- Ne možemo instancirati sealed klase, samo potklase
- Korisne kod korištenja when izraza.

```
sealed class HomeScreenState

object Initial : HomeScreenState()

data class Loading(val progress: Float) : HomeScreenState()

data class Ready(val data: List<String>) : HomeScreenState()

data class Error(
    val exception: Exception,
    val message: String
) : HomeScreenState()
```

```
val state = getHomeScreenState()
  when (state) {
      is Error -> println(state.message) // Smart cast to Error
      Initial -> {
          // start an operation
      }
      is Loading -> showProgress(state.progress) // Smart cast
      is Ready -> showItems(state.data) // Smart cast
   }
}
```

Delegacija

Delegation

Delegacija ponašanja klase

Delegirana svojstva

Delegacija ponašanja klase

```
interface Engineer {
    fun buildApp()
    fun testApp()
    fun writeNewBackendApi()
}
```

Delegacija ponašanja klase

```
class SeniorEngineer : Engineer

class Manager : Engineer
```

```kotlin
class SeniorEngineer : Engineer {
    override fun buildApp() {
        println("I'm building a new Android app!")
    }

    override fun testApp() {
        println("No bugs found! Ready to launch...")
    }

    override fun writeNewBackendApi() {
        println("Creating user endpoints.")
    }
}
```

```kotlin
class Manager(engineer: Engineer) : Engineer by engineer
{
    fun organizeMeeting() {
        println("Let's have a sprint planning!")
    }
}


val filip = SeniorEngineer()
val pero = Manager(engineer = filip)
pero.organizeMeeting()
pero.buildApp()
```

Delegirana svojstva

- Lazy svojstva
  - Vrijednost se izračunava kod prvog pristupa

```
val expensiveObject by lazy { produceExpensiveObject() }
```

Delegirana svojstva

- Observable svojstva
  - Poziva se na svaku promjenu vrijednosti

```
var name by Delegates.observable("Filip") { _, oldValue,
newValue ->
   println("Old: $oldValue, new: $newValue")
}
name = "Luka"
```

```
Old: Filip, new: Luka
```

Delegated properties

- Map
  - Store properties in a Map

```
val nameToJob = mapOf(
    "filip" to "Compose Dev",
    "luka" to "Android Dev"
)
```

```
val filip by nameToJob
println(filip)
```

Output: Compose Dev

Lateinit

- Inicijalizacija var varijable kasnije.
- Možemo koristiti u sljedećim slučajevima:
  - Svojstvo definirano unutar klase.
  - Bez custom get/set pristupa.
  - Lokalne i top-level varijable.
  - Non-null, non-primitive tipovi.

```
lateinit var course: Course

if(::course.isInitialized) {
  println(course.name)
}
```

Hvala na pažnji!

Pitanja?